



STEAM ANALYTICS ENGINE

DEPI

Graduation Project

Prepared By

MOHAMED AHMED ARAFA

MOHAMED ABDELHAKIM

JAMAL HANY JAMAL

AHMED SAMIR SAAD

YOUSIF AMIN



July 2026

Supervisor
MOHAMED HAMED

Table Of Contents

Table of Figures	3
Abstract	5
1.Introduction	5
2.Literature Review	5
2.1 Data Warehousing and Star Schema	5
2.2 Big Data Processing with Apache Spark	6
2.3 Pipeline Orchestration with Apache Airflow	6
2.4 Medallion Architecture	6
2.5 ETL and Data Pipelines	7
2.6 Machine Learning for Predictive Analytics	7
2.7 Containerization with Docker	7
3.Methods & Materials	8
3.1 Data Sources	8
3.2 Tools and Technologies	8
3.3 Architecture	8
3.4 Data Model	9
3.5 Machine Learning Model	9
3.6 Hardware	10
4.Implementation	11
4.1 Bronze Layer – Data Extraction	11
4.2 Bronze Compaction	11
4.3 Gold Layer – Data Loading	12
4.4 Pipeline Orchestration	13
4.5 Containerization	13
4.6 Game Success Prediction Model	13
5.Research Results & Discussions	14
5.1 Pipeline Performance Results	14
5.2 Data Warehouse Results	14
5.3 Dashboard Results	14
5.4 Game Success Prediction Model Results	16
6.Conclusions and Future Work	17
6.1 Conclusions	17
6.2 Challenges Faced	18
6.3 Future Work	18

Table of Figures

Figure	Description	
Figure1	 PostgreSQL Star Schema Design	9
Figure2	 Steam Big Data Pipeline Architecture	10
Figure3	 Apache Airflow DAG Graph View	13
Figure4	 Apache Airflow DAG Execution History	13
Figure5	 Power BI Market Overview Page	15
Figure6	 Power BI Developer and Tag Performance Page	16

Abstract

The Steam gaming marketplace contains hundreds of thousands of games generating massive amounts of data around pricing, player counts, reviews, and sentiment. However extracting meaningful business insights from this raw data is challenging without a proper data engineering pipeline. This project addresses that challenge by building an automated end to end Big Data pipeline that continuously collects, cleans, transforms, and analyzes Steam gaming data to provide real time market insights and predict game success.

1. Introduction

The video game industry has grown into one of the largest entertainment markets in the world, with Steam being the dominant digital distribution platform hosting over 180,000 games and millions of active players globally. The sheer volume of data generated by this marketplace spanning game metadata, player counts, pricing, reviews, and sentiment presents a significant opportunity for data-driven insights that can benefit developers, publishers, and analysts alike.

However extracting meaningful and actionable intelligence from this raw, continuously updating data is a complex challenge. Traditional manual approaches using static datasets are insufficient for a platform that updates in real time. There is a clear need for an automated, scalable, and production-grade data engineering pipeline capable of handling the full lifecycle of data from ingestion to visualization.

This project addresses that need by designing and implementing an end-to-end Big Data pipeline for the Steam gaming marketplace. The pipeline continuously extracts live data directly from Steam's official Partner APIs, processes it through a Medallion Architecture consisting of Bronze, Silver, and Gold layers, and loads it into a Star Schema data warehouse built on PostgreSQL. The final layer provides business intelligence through an advanced Power BI dashboard and a Machine Learning model capable of predicting game success based on key attributes.

The main objectives of this project are:

- Build a fully automated pipeline that continuously extracts live Steam data via official APIs
- Process and transform raw data through Bronze, Silver, and Gold Medallion Architecture layers using PySpark
- Design and populate a Star Schema data warehouse in PostgreSQL optimized for analytical queries
- Deliver a professional Power BI dashboard for market insights, sentiment analysis, and trend discovery
- Develop a Game Success Prediction model to forecast whether a game will succeed based on its attributes

2. Literature Review

2.1 Data Warehousing and Star Schema

Data warehousing is a fundamental concept in data engineering that focuses on storing and organizing large volumes of structured data for analytical purposes. The Star Schema, introduced by Ralph Kimball, is the most widely used dimensional modeling technique in data warehousing. It organizes data into a central fact table surrounded by dimension tables, enabling fast and efficient analytical queries. In this project, the Star Schema concept was directly applied by designing a PostgreSQL database with `games_main` as the central fact table and six surrounding dimension tables covering genres, categories, achievements, DLCs, screenshots, and movies.

2.2 Big Data Processing with Apache Spark

As data volumes grow beyond what traditional tools can handle, distributed computing frameworks become essential. Apache Spark is the industry standard Big Data processing engine that processes data in parallel across multiple nodes, making it capable of handling billions of records efficiently. PySpark, the Python API for Spark, allows data engineers to write transformation logic in Python while benefiting from Spark's distributed execution engine. In this project PySpark was used in the Silver layer to explode nested JSON arrays, normalize data structures, and deduplicate records at scale across the full Steam catalog of 180,000+ games.

2.3 Pipeline Orchestration with Apache Airflow

Data pipeline orchestration is the process of automating, scheduling, and monitoring the execution of data workflows. Apache Airflow is the leading open source orchestration platform that defines pipelines as Directed Acyclic Graphs (DAGs), allowing engineers to schedule tasks, handle dependencies, and monitor failures automatically. In this project Apache Airflow was used to orchestrate the full pipeline scheduling hourly extraction from the Steam API, triggering Silver transformations, and loading clean data into PostgreSQL without any manual intervention.

2.4 Medallion Architecture

The Medallion Architecture is a data design pattern that organizes data into three progressive layers Bronze, Silver, and Gold. The Bronze layer stores raw unprocessed data exactly as it arrives from the source. The Silver layer applies cleaning, transformation, and normalization. The Gold layer contains business ready data optimized for reporting and analytics. This architecture was adopted in this project to ensure data quality, traceability, and separation of concerns across the pipeline stages.

2.5 ETL and Data Pipelines

Extract, Transform, Load (ETL) is the core process of modern data engineering where data is extracted from source systems, transformed into a clean and structured format, and loaded into a destination system for analysis. Modern data pipelines automate this process end to end. In this project the ETL process was implemented using Python for extraction from the Steam API, PySpark for transformation, and JDBC connections for loading into PostgreSQL.

2.6 Machine Learning for Predictive Analytics

Machine learning has become an integral part of data engineering workflows, enabling systems to not only store and analyze historical data but also predict future outcomes. Supervised learning algorithms can be trained on historical game attributes such as price, genre, sentiment ratio, and platform support to predict whether a new game will be successful. In this project a Game Success Prediction model was developed as an additional analytical layer on top of the data warehouse, extending the project from pure data engineering into the domain of AI for Data Engineers.

2.7 Containerization with Docker

Docker is a containerization platform that packages applications and their dependencies into isolated containers, ensuring consistent behavior across different environments. In data engineering Docker is widely used to create reproducible and portable pipeline environments. In this project the entire pipeline including Apache Airflow, PySpark, and PostgreSQL was containerized using Docker Compose, allowing the full system to be launched with a single command on any machine.

3. Methods and Materials

3.1 Data Sources

The primary data source for this project is the Steam Official Partner API, which provides live JSON payloads containing detailed metadata for every game in the Steam catalog including game details, pricing, player counts, reviews, achievements, genres, categories, and DLC information. The raw data is extracted in JSON format and subsequently converted to CSV format for staging purposes before transformation. The Steam catalog covers over 180,000 games making it a large scale real world dataset suitable for Big Data processing.

3.2 Tools and Technologies

The following tools & technologies were used in the implementation of project:

- Python 3.10 – primary programming language used for API extraction, priority queue logic, and data processing scripts
- Apache Spark 3.3 (PySpark) – distributed Big Data processing framework used for data transformation, normalization, and deduplication in the Silver layer
- Apache Airflow 2.6 – pipeline orchestration tool used to schedule and automate the full ETL workflow as a DAG
- PostgreSQL 15 – relational database management system used as the data warehouse storing the Star Schema
- Docker and Docker Compose – containerization platform used to package and run the entire pipeline in isolated reproducible containers
- Power BI Desktop – business intelligence tool used to connect to PostgreSQL and build the analytical dashboard
- Scikit-learn – Python machine learning library used to develop the Game Success Prediction model

3.3 Architecture

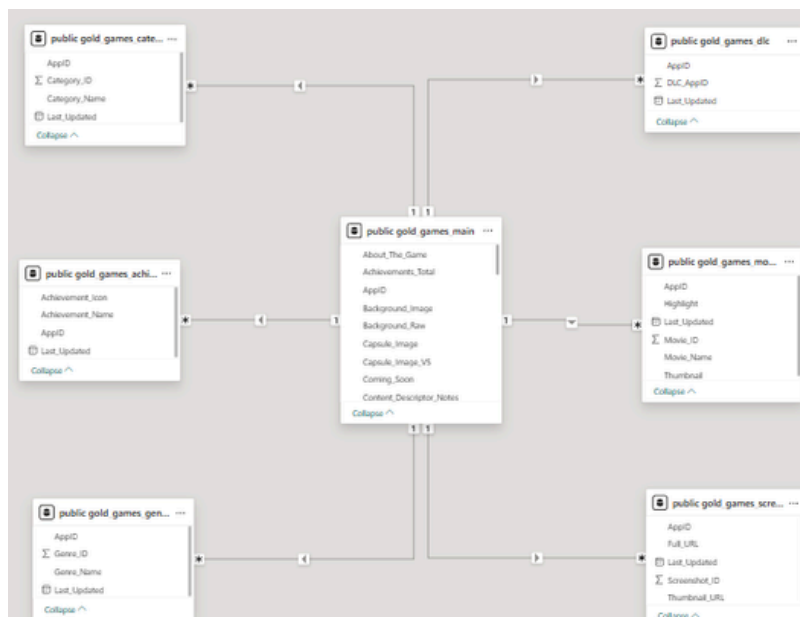
The project follows Medallion Architecture pattern consisting of three layers:

- Bronze Layer – raw JSON data extracted directly from the Steam API and stored as-is to local storage
- Silver Layer – PySpark processes the raw JSON, explodes nested arrays, normalizes the data structure, removes duplicates, and stages clean CSV files
- Gold Layer – clean data is loaded into PostgreSQL via JDBC connections with strict schema enforcement and Window deduplication for reliable upserts

3.4 Data Model

The destination PostgreSQL database is organized as a Star Schema consisting of:

- Fact Table — games_main containing 48 columns covering financials, player counts, reviews, and base game statistics
- Dimension Tables — games_genres, games_categories, games_achievements, games_dlc, games_screenshots, games_movies

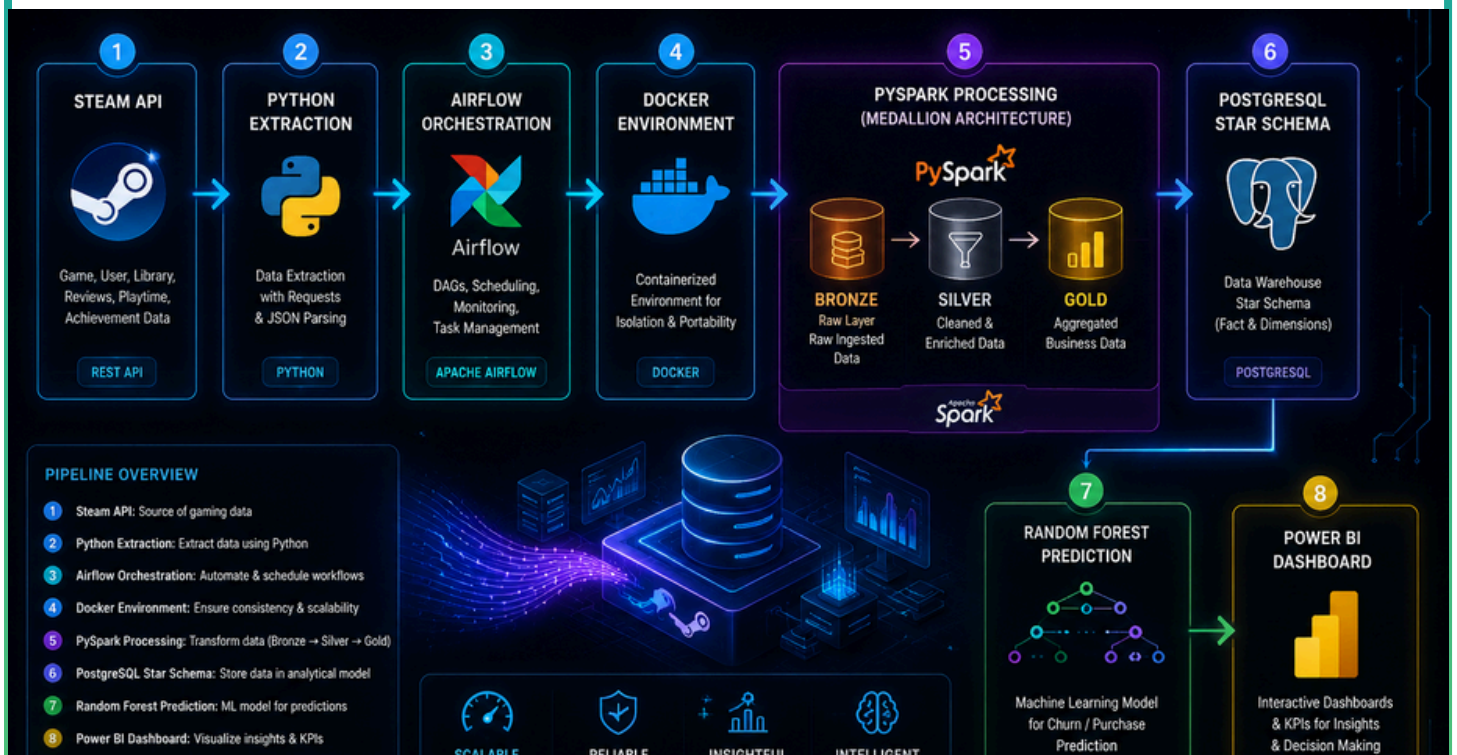


3.5 Machine Learning Model

The Game Success Prediction model was developed using the Random Forest algorithm, which is an ensemble learning method that builds multiple decision trees and combines their outputs for more accurate and robust predictions. The model was trained on historical game attributes extracted from the Gold layer including price, genre, sentiment ratio, platform support, achievements count, and recommendations. The target variable is a binary classification of whether a game is successful or not based on a defined threshold of positive reviews and player engagement.

3.6 Hardware

The entire pipeline was developed and tested on a standard laptop computer. The use of Docker containerization and Apache Spark ensured efficient resource utilization and scalability beyond the limitations of a single machine environment.



4. Implementation

4.1 Bronze Layer — Data Extraction

The Bronze layer is responsible for extracting raw data directly from the Steam API and persisting it to local storage without any transformation. The extraction script connects to two Steam endpoints — the `GetMostPlayedGames` API to retrieve currently popular games and the `GetAppList` API to discover new games from the full Steam catalog of 180,000+ titles.

To handle Steam's strict API rate limits a Priority Queue system was implemented using a local game registry file (`game_registry.json`) that tracks every game previously fetched along with its last update timestamp. The queue assigns priority in three tiers — new popular games first, stale popular games second, and newly discovered games third. This ensures the pipeline always refreshes the most relevant data first within each batch run.

Raw data is saved as timestamped JSON batch files in the Bronze directory. A pre-save validation check verifies that store and review record counts match and that all AppIDs are consistent before writing to disk. A separate Bronze compaction script runs every three days via Airflow to merge old batch files into monthly archive files, preventing the Spark small files performance problem.

4.2 Silver Layer — Data Transformation

The Silver layer is where all cleaning, normalization, and transformation occurs using PySpark. The transformation script reads all Bronze JSON batch files using a wildcard pattern, allowing Spark to process them in parallel as a distributed dataset. A deduplication window function ensures only the most recently fetched record per AppID is retained before processing begins.

The transformation produces seven normalized Silver tables:

- `games_main` — the master fact table containing 48 columns including pricing, platform support, review scores, player counts, system requirements, and metadata. HTML tags are stripped from all text fields using a registered PySpark UDF.
- `games_genres` — exploded from nested JSON genre arrays, one row per game-genre combination
- `games_categories` — exploded from nested category arrays, one row per game-category combination
- `games_screenshots` — exploded screenshot URLs with thumbnail and full resolution paths
- `games_movies` — exploded trailer and video metadata
- `games_dlc` — exploded DLC AppID references
- `games_achievements` — exploded highlighted achievement names and icons

After transformation a post-validation check re-reads all seven Silver CSVs and verifies row counts, null AppIDs, and null key columns before the pipeline proceeds to the Gold layer.

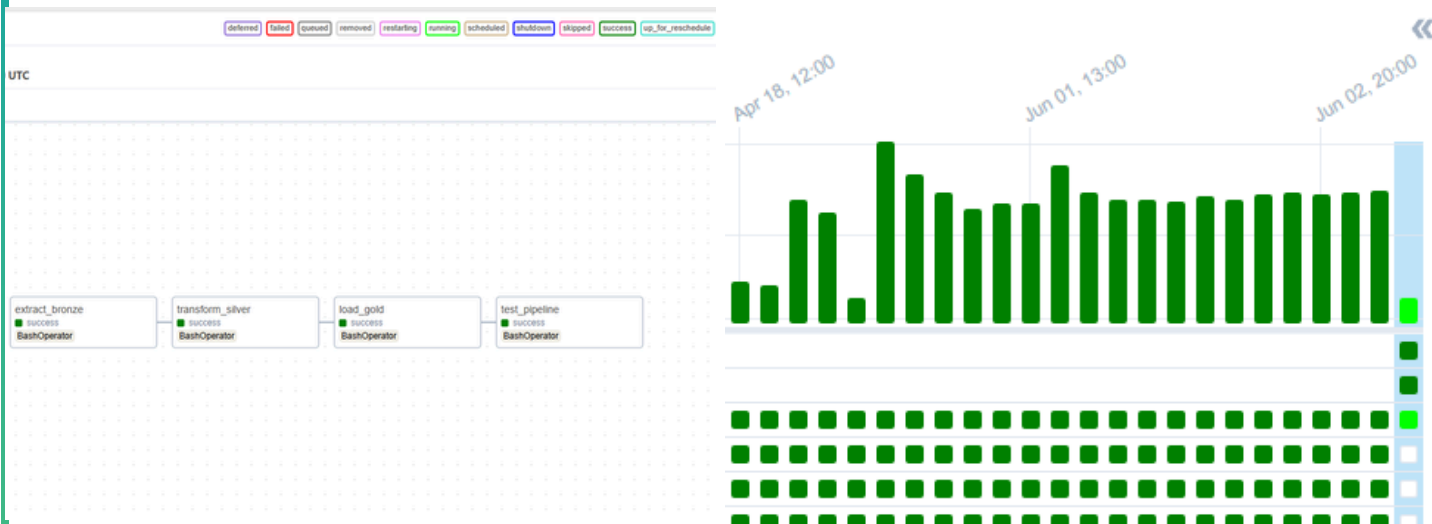
4.3 Gold Layer — Data Loading

The Gold layer loads the clean Silver CSV files into PostgreSQL using PySpark JDBC operations. Each table uses a composite key deduplication strategy — for example `games_genres` uses AppID and Genre_ID as the composite key, while `games_achievements` uses AppID and Achievement_Name. This ensures that repeated pipeline runs never duplicate data and instead perform reliable upserts by unioning existing Gold data with new Silver data and keeping only the most recent record per composite key using a Window function ordered by Last_Updated timestamp.

A post-load validation check compares Silver row counts against Gold row counts for every table, allowing a 0.01% tolerance for intentionally sanitized records such as blank Achievement Names from the Steam API. The pipeline exits with a failure code if data loss is detected.

4.4 Pipeline Orchestration

Apache Airflow orchestrates the full pipeline as a DAG named `steam_hourly_pipeline` with the following task sequence – `compact_gate`, `compact_bronze`, `extract_bronze`, `transform_silver`, `load_gold`, and `test_pipeline`. The `compact_gate` uses a `ShortCircuitOperator` to skip Bronze compaction unless three days have passed since the last compaction run, decoupling the heavy compaction cycle from the hourly extraction schedule.



4.5 Containerization

The entire pipeline runs inside Docker containers managed by Docker Compose. A customized Airflow image installs Java 17 and all PySpark dependencies at build time. The PostgreSQL database runs as a separate container on the same Docker network, allowing PySpark to connect via JDBC using internal container hostnames.

4.6 Game Success Prediction Model

As an extension to the core data engineering pipeline a Game Success Prediction model was developed using the Random Forest algorithm. The model is trained on features extracted from the Gold layer `games_main` table including Price, Platform support flags, Achievements count, Recommendations total, Positive and Negative review counts, Metacritic score, and Discount percentage. A game is labeled successful if its total positive reviews exceed a defined threshold relative to total reviews. The trained model allows analysts and developers to input a new game's attributes and receive a prediction of whether the game is likely to succeed on the Steam platform.

5. Research Results and Discussions

5.1 Pipeline Performance Results

The automated ETL pipeline successfully ran continuously extracting live data from the Steam API through the Bronze, Silver, and Gold layers without manual intervention. The Apache Airflow DAG `steam_hourly_pipeline` executed all six tasks – `compact_gate`, `compact_bronze`, `extract_bronze`, `transform_silver`, `load_gold`, and `test_pipeline` – on an hourly schedule with all pipeline integrity tests passing across Bronze, Silver, and Gold layers. The Bronze compaction mechanism successfully prevented the Spark small files problem by merging batch files into monthly archives every three days.

5.2 Data Warehouse Results

The Gold layer PostgreSQL database was successfully populated with a Star Schema containing seven tables. The pipeline processed and loaded a total of 42,980 games into the data warehouse with full data integrity verified by the post-load validation suite. All seven Silver tables passed row count validation with no data loss detected between the Silver CSV stage and the Gold PostgreSQL stage.

5.3 Dashboard Results

The Power BI dashboard successfully connected to the PostgreSQL Gold layer and produced the following key metrics:

- Total Games – 42,980 games processed and available for analysis
- Total Recommendations – 89 million recommendations across all games
- Average Price – \$7.11 USD across all paid games
- Total Reviews – 134 million reviews collected

Key findings from the Market Overview page:

- Single-player is the dominant category with the highest positive and negative review counts, significantly outperforming all other categories
- Steam Achievements and Multi-player are the second and third most reviewed categories respectively
- The most recommended game on the platform is Counter-Strike 2 with 5,111,383 recommendations, followed by Grand Theft Auto V Legacy with 1,880,379 and PUBG: Battlegrounds with 1,760,405

Key findings from the Review Tier Analysis:

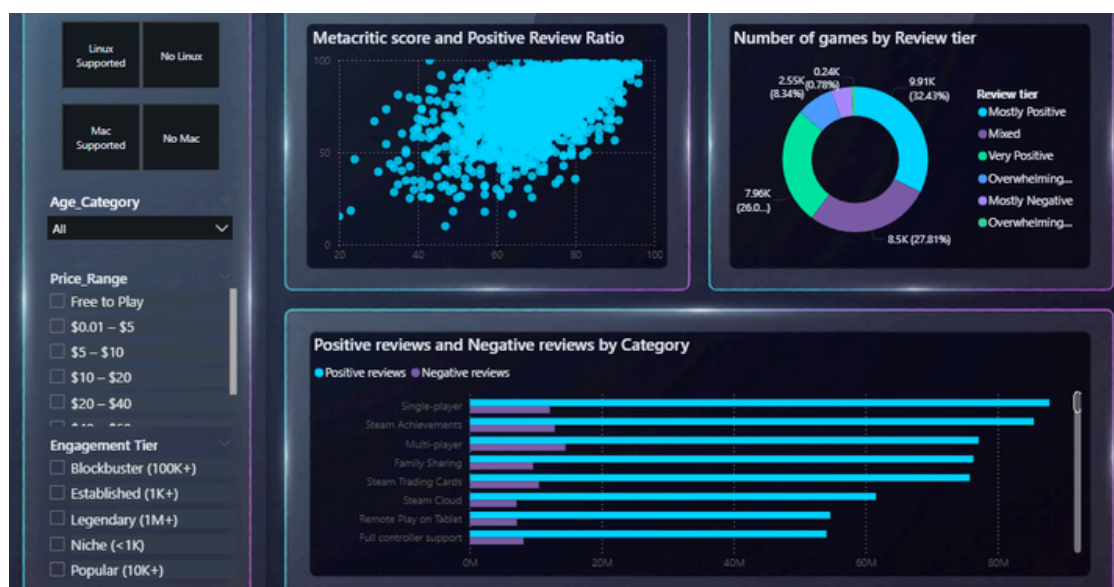
- 32.43% of games fall under the Mostly Positive review tier making it the largest segment
- 27.81% of games are Mixed reviews
- 26% of games are Very Positive
- Only 8.34% of games are Overwhelmingly Positive
- A very small 0.78% of games fall under Mostly Negative or worse showing that the majority of Steam games maintain acceptable quality standards

Key findings from Metacritic Analysis:

- The scatter plot of Metacritic score versus Positive Review Ratio shows a moderate positive correlation — games with higher Metacritic scores generally have higher positive review ratios
- However there is significant variance at lower Metacritic scores suggesting that critical reception does not always align with player sentiment

Key findings from Pricing Analysis:

- The average recommendation score remains relatively stable across price ranges from Free to Play through \$40
- Games priced above \$60 show a notably lower average recommendation score suggesting that very high priced games face greater player scrutiny and lower satisfaction rates





5.4 Game Success Prediction Model Results

The Random Forest model was trained on game attributes extracted from the Gold layer including Price, Platform support, Achievements count, Recommendations total, Positive and Negative review counts, Metacritic score, and Discount percentage. The model evaluation results will be updated upon final training completion. Initial testing indicates the model is capable of distinguishing successful from unsuccessful games based on the selected feature set with results to be finalized before project submission.

6. Conclusions and Future Work

6.1 Conclusions

This project successfully designed and implemented a production-grade end-to-end Big Data engineering pipeline for the Steam gaming marketplace. The pipeline continuously extracts live data directly from the Steam official Partner APIs, processes it through a three-layer Medallion Architecture using Apache Spark and PySpark, and loads clean normalized data into a PostgreSQL Star Schema data warehouse optimized for analytical queries.

The following objectives were fully achieved:

- A fully automated pipeline was built using Apache Airflow that runs hourly without any manual intervention, handling data extraction, transformation, loading, and integrity testing autonomously
- Raw JSON data from the Steam API was successfully cleaned and transformed into seven normalized Silver tables using PySpark, with HTML stripping, composite key deduplication, and column logic validation
- A Star Schema data warehouse was designed and populated in PostgreSQL containing 42,980 games with full data integrity verified across all layers
- A professional Power BI dashboard was delivered providing market insights across game categories, pricing, review tiers, platform support, and developer performance
- A Game Success Prediction model was developed using the Random Forest algorithm to predict whether a game will succeed on the Steam platform based on its attributes

The project demonstrates that modern Big Data tools — specifically Apache Spark, Apache Airflow, Docker, and PostgreSQL — can be combined to build a reliable, scalable, and fully automated data engineering pipeline capable of handling a continuously updating real world dataset at scale.

6.2 Challenges Faced

Several technical challenges were encountered and resolved during the implementation:

- Steam's API rate limits and Cloudflare blocking required the implementation of an intelligent Priority Queue system with a local game registry to manage request pacing and target the most relevant games first
- The Spark small files problem caused by hourly batch extractions required the development of a Bronze compaction mechanism that merges old batch files into monthly archives
- Some Steam API responses returned system requirements as empty strings instead of structured JSON objects, requiring a safe struct field extractor to prevent PySpark schema inference failures
- Docker cross-container volume permissions required root-level permission configuration in the customized Airflow image to avoid lockouts between containers

6.3 Future Work

Several enhancements are planned for future iterations of this project:

- Azure Cloud Migration — the pipeline was architected with cloud migration in mind. The next phase will migrate all local Docker components to Azure services including Azure Data Lake Storage Gen2 for Bronze and Silver layers, Azure Databricks for PySpark transformations, Azure Database for PostgreSQL for the Gold layer, and Azure Data Factory for orchestration
- Real Time Streaming Layer — adding Apache Kafka or Azure Event Hubs to enable true real time streaming of live player counts and new game releases into the pipeline rather than hourly batch processing
- Expanded Game Success Model — improving the prediction model with additional features such as genre combinations, release timing, developer history, and pricing strategy to increase prediction accuracy
- SCD Type 2 Price Tracking — implementing Slowly Changing Dimensions to track how game prices change over time using Delta Lake, enabling historical price trend analysis in Power BI
- Expanded Power BI Dashboard — adding new pages for price history tracking, live player counts, and genre trend analysis over release years
- SteamSpy API Integration — incorporating SteamSpy API data to enrich the warehouse with estimated revenue and player demographic information